

Lab 01: what the same job costs in English, Russian and Kazakh

Student: Akmurzina Mariza

1. Prediction vs. measurement

Prediction (written before Part 2). I predict the number of tokens for the whole request (system prompt + complaint) sent to Claude in English, Russian and Kazakh. English will be the cheapest and Kazakh the most expensive. Kazakh / English: about 4 times. Russian / English: about 2 times. I think Claude behaves closer to the harsh tokenizer cl100k_base, because I assume that Kazakh is rare in the training data, so the tokenizer splits it into small pieces. Kazakh will be longer than Russian because the tokenizer was trained mostly on English text, and I expect it to handle the Kazakh-specific letters (ә, Ғ, Қ, Ң, Ө, Ұ, Ү, Һ, І) worst of all. The final bill will differ from the token ratio of the request, because it also depends on the length of the answer, and answer tokens cost more.

Token ratio	My prediction (written before Part 2)	Measured value	Source
Kazakh / English	~4x	2.35x	own run*
Russian / English	~2x	1.30x	own run*

*Input-token ratio of the whole request (system prompt + complaint), billed by Google on gemini-3.5-flash-lite in my own Part 2 run (Google Colab). My prediction was written for ClaudeI measured Gemini, because I had no Claude API key.

My prediction was inaccurate: I expected Kazakh about 4 times and Russian about 2 times more tokens than English, but the measurement showed 2.35x and 1.30x. I got the order of the languages right. The total bill differs from the token ratio, as I wrote at the end of my prediction: Kazakh costs 3.78 times and Russian 1.75 times the English bill, because the Kazakh answer was longer (161 output tokens against 38) and output tokens cost more. The 3.78x is a different quantity from the predicted token ratio, so it does not confirm the prediction.

2. Annual cost

Volume: 300 requests/day (109,500 per year). Justification: I assume a small financial company with 30,000 customers, where about 1% contact support every day, which gives 300 requests per day.

Model (USD per year)	English	Russian	Kazakh
gemini-3.1-flash-lite	\$9	\$15	\$33
gemini-3.5-flash-lite	\$14	\$24	\$52
gemini-3.8-flash	\$24	\$40	\$86
gemini-3.1-pro-preview	\$72	\$123	\$263

Source: part3_cost.py --requests-per-day 300 on my own Part 2 run (tokens and answer lengths billed on gemini-3.5-flash-lite, applied to all four models); prices from ai.google.dev/gemini-api/docs/pricing, checked 2026-09-21; gemini-3.8-flash prices double on 2027-01-01.

3. Model for a Kazakh-language support queue

Chosen model: gemini-3.5-flash-lite.

Cost argument: For Kazakh at 300 requests/day it costs \$52 per year (gemini-3.1-flash-lite \$33, gemini-3.8-flash \$86, gemini-3.1-pro-preview \$263). The differences are small at this volume, and it is the only model whose cost I measured on its own answers; the other rows assume the same answer length.

Quality argument: The complaint mentions an attached contract and statement, but nothing is attached. In my run this model answered in all three languages without inventing a reason for the rate change: it said it had no such information and named a next step (provide the documents, or in Kazakh contact a bank branch). This is one answer per language on one model, enough for a pilot; before production it needs a test on many more complaints. gemini-3.8-flash could not be tested (error 503, server overloaded).

4. A cost lever this lab did not use

A cost lever this lab did not use is shortening the answers: output tokens are priced 5 to 8 times higher than input tokens on the four Gemini models, so asking the model to answer in at most two sentences would cut the bill more than any shortening of the request.

AI-use declaration

Tools used

Claude (Anthropic), and the Gemini API (gemini-3.5-flash-lite) as the model measured in Part 2

What I used it for

To suggest the wording template of my prediction (its structure); to correct small wording errors in my prediction and translate it into English; to adapt the lab scripts (prices.py, part2_measure.py and four lines of part3_cost.py) from the Claude API to the Gemini API.

What I did myself

I created my own Gemini API key, ran the scripts in Google Colab and took the token counts, answer lengths and costs from their output, chose the numbers and the hypothesis of my Part 1 prediction myself before seeing any measured values, chose the scenario and the volume myself, chose the cost lever.

What I did not use AI for

Producing the token counts, answer lengths and costs (they come from my own run of the scripts on the Gemini API), creating the API key, choosing the scenario and the volume, and choosing the prediction numbers.